

BAD CODE

Bird

```
public interface Bird {  
    void eat();  
    void fly();  
}
```

Sparrow

```
public class Sparrow implements Bird{  
    @Override  
    public void eat() {  
        System.out.println("Sparrow is eating seeds");  
    }  
  
    @Override  
    public void fly() {  
        System.out.println("Sparrow is flying high in the sky");  
    }  
}
```

Ostrich

```
public class Ostrich implements Bird {  
    @Override  
    public void eat() {  
        System.out.println("Ostrich is eating grass");  
    }  
  
    @Override  
    public void fly() {  
        throw new UnsupportedOperationException("Ostrich can't fly");  
    }  
}
```

Eagle

```
public class Eagle implements Bird{  
    @Override  
    public void eat() {  
        System.out.println("Eagle is eating a fish");  
    }  
  
    @Override  
    public void fly() {  
        System.out.println("Eagle is soaring over the mountains");  
    }  
}
```

BAD CODE

BirdSanctuary

```
public class BirdSanctuary {  
    public void feed(Bird bird) {  
        bird.eat();  
    }  
  
    public void makeItFly(Bird bird) {  
        bird.fly();// Not every bird can fly  
    }  
}
```

BAD CODE

Client

```
public class Client {  
  
    public static void main(String[] args) {  
        BirdSanctuary sanctuary = new BirdSanctuary();  
  
        Sparrow sparrow = new Sparrow();  
        Eagle eagle = new Eagle();  
        Ostrich ostrich = new Ostrich();  
  
        System.out.println("----- Feeding individual birds -----");  
        sanctuary.feed(sparrow);  
        sanctuary.feed(eagle);  
        sanctuary.feed(ostrich);  
  
        System.out.println();  
  
        System.out.println("----- Making individual birds fly -----");  
        sanctuary.makeItFly(sparrow);  
        sanctuary.makeItFly(eagle);  
        sanctuary.makeItFly(ostrich); // CRASH  
    }  
}
```

Code will crash
as Ostrich can not
fly



ummm i can not fly

and you will see a 

GOOD CODE

Bird

```
public interface Bird {  
    void eat();  
}
```

FlyingBird

```
public interface FlyingBird extends Bird {  
    void fly();  
}
```

Sparrow

```
public class Sparrow implements FlyingBird{  
    @Override  
    public void eat() {  
        System.out.println("Sparrow is eating seeds");  
    }  
  
    @Override  
    public void fly() {  
        System.out.println("Sparrow is flying high in the sky");  
    }  
}
```

Eagle

```
public class Eagle implements FlyingBird{  
    @Override  
    public void eat() {  
        System.out.println("Eagle is eating a fish");  
    }  
  
    @Override  
    public void fly() {  
        System.out.println("Eagle is soaring over the mountains");  
    }  
}
```

Ostrich

```
public class Ostrich implements Bird {  
    @Override  
    public void eat() {  
        System.out.println("Ostrich is eating grass");  
    }  
}
```

GOOD CODE

BirdSanctuary

```
public class BirdSanctuary {  
  
    public void feed(Bird bird) {  
        bird.eat();  
    }  
  
    public void makeItFly(FlyingBird flyingBird){  
        flyingBird.fly();// Not every bird can fly  
    }  
}
```

GOOD CODE

Client

```
public class Client {  
  
    public static void main(String[] args) {  
        BirdSanctuary sanctuary = new BirdSanctuary();  
  
        Sparrow sparrow = new Sparrow();  
        Eagle eagle = new Eagle();  
        Ostrich ostrich = new Ostrich();  
  
        System.out.println("----- Feeding individual birds -----");  
        sanctuary.feed(sparrow);  
        sanctuary.feed(eagle);  
        sanctuary.feed(ostrich);  
  
        System.out.println();  
  
        System.out.println("----- Making individual birds fly -----");  
        sanctuary.makeItFly(sparrow);  
        sanctuary.makeItFly(eagle);  
  
        sanctuary.makeItFly(ostrich); //CE  
  
    }  
}
```

IMP

Now the compiler prevents runtime crashes.



Finally,
they are not forcing me
to fly.
I'll happily continue
my habits –
I have more weight!